MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión B del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_gonzalez_luis
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/gonzalez_luis/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
En la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

Tu respuesta

```
El atributo es una caracteristica del objeto, es algo que lo define
El metodo es lo que puede hacer dicho objeto
Por ejemplo un objeto llamado "carro" puede tener como atributo "color" y como metodo
"avanzar"
```

Corrección. Correcta: distingue el atributo como característica que define al objeto (estado) y el método como lo que puede hacer (comportamiento), con ejemplos coherentes (color y avanzar en un carro).

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

Tu respuesta

```
main asigna la "prioridad" de una parte del codigo, lo que se asigna como main es lo primero
que se ejecuta al probar un programa.
Se usa main para declarar que no tendrá cambios dentro del codigo
```

Corrección. Reconoce que main es lo primero que se ejecuta (punto de entrada), aunque hablar de 'prioridad' es impreciso. La explicación de static es incorrecta: no significa que no tendrá cambios, sino que main se puede invocar desde la clase sin crear antes una instancia.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

Tu respuesta

```
Un atributo static se le asigna a cada objeto que se crea dentro de la clase y es invariable,
mientras que un atributo de instancia puede cambiar de un objeto a otro.
Por ejemplo siguiendo con los carros, un atributo static puede ser que todos deben tener 4
ruedas mientras que un atributo de instancia puede ser el color, todos tienen un color pero no
es el mismo para todos
```

Corrección. Acierta en que el atributo de instancia puede variar de un objeto a otro (el color) y el ejemplo de las 4 ruedas sugiere un dato común. Pero hay un error de concepto: un atributo static no es invariable ni se asigna a cada objeto. Pertenece a la clase, existe una sola copia que comparten todas las instancias y su valor puede cambiar, como un contador de objetos creados.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Tu respuesta

```
Una variable de tipo primitivo (int, double, char, etc.) guarda un tipo determinado de dato.
Mientras que una variable de referencia guarda la direccion de memoria
Un ejemplo de variable de tipo primitivo podria ser un int = 5
```

Corrección. Correcto en lo esencial: el primitivo guarda el dato (el valor) y la referencia guarda la dirección o referencia a un objeto, con el ejemplo int = 5. Falta el ejemplo de una variable de referencia (por ejemplo, un String o un arreglo).

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

Tu respuesta

```
Cuando usas Javac compila un archivo. java y con esto se crea un archivo ejecutable tipo.class
```

Corrección. Bien que javac compila el .java y produce un .class, aunque ese archivo es bytecode, no un ejecutable nativo. Falta la segunda parte: la JVM carga y ejecuta ese bytecode.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[] datos = {7, 5, 3};
int[] alias = datos;
int i = 0;
while (i < 3) {
    System.out.println(alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println(mitad);
```

Tu respuesta

```
3
2
1
B79
2.5
```

Corrección. Las líneas 3, 2, 1 y B79 son correctas. La última es 2.0, no 2.5, porque datos[1] / 2 es división entera (5/2 = 2) y el cast a double se aplica después, sobre el resultado. El formato decimal es correcto.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int modo = 1;
switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
LISTO
```

Corrección. La clasificación 'Funciona' es correcta. La salida está incompleta: imprime LISTO / FIN, porque sin break la ejecución sigue al case siguiente; faltó explicar la regla.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
{ int nivel = 4; }
System.out.println(nivel);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
La sintaxis para nivel es erronea
```

Corrección. La clasificación 'No compila' es correcta. La causa apunta a nivel pero no es un error de sintaxis: es de ámbito, porque nivel solo existe dentro de las llaves que la declaran y fuera de ellas no se reconoce.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
etiqueta es una variable que esta vacia, por lo que no podrá imprimir nada
```

Corrección. La clasificación 'Compila pero truena' es correcta. La causa va bien encaminada, pero etiqueta no está 'vacía': vale null y llamar trim() sobre null lanza una NullPointerException al no haber objeto.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

Sin respuesta.

Justificaciones

Sin respuesta.

> **Corrección.** Sin respuesta.

R4 · Código con repetición

Pregunta

El arreglo intentos contiene exactamente tres códigos enteros, en el orden en que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null. No lo modifiques.

```
static int intentoValido(int[] intentos, int clave)
```

Ejemplos (argumentos → retorno):

{13, 42, 8}, 42 → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3}, 42 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Devuelve la acción del módulo con switch sobre Modo. Primero: si bateria < 10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

```
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
static String accion(Modo modo, int bateria)
```

Regla o valor recibido	Texto devuelto
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTAR"
PRUEBA	"SIMULAR"

Tu respuesta

```
static String accion (Modo modo, int bateria) {
    if (bateria < 10)
        System.out.println("CARGAR");
    else
        System.out.println("SIN MODO");

    switch (Modo){
        case 1: System.out.println("DETENER");
        case 2: System.out.println("LEER");
        case 3: System.out.println("AJUSTAR");
        case 4: System.out.println("SIMULAR");
    }
```

Corrección. Cabecera correcta, usa switch y la comprobación bateria < 10 va primero. Fallas: imprime en lugar de devolver con return; el else da 'SIN MODO' a cualquier modo con batería suficiente en vez de revisar modo == null, y el switch se ejecuta siempre y cae de un case al siguiente. Además usa switch(Modo) con case 1 a 4 en lugar de switch(modo) con case APAGADO, MEDIR, etc. (los textos sí están emparejados en el orden correcto) y falta la llave de cierre.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.